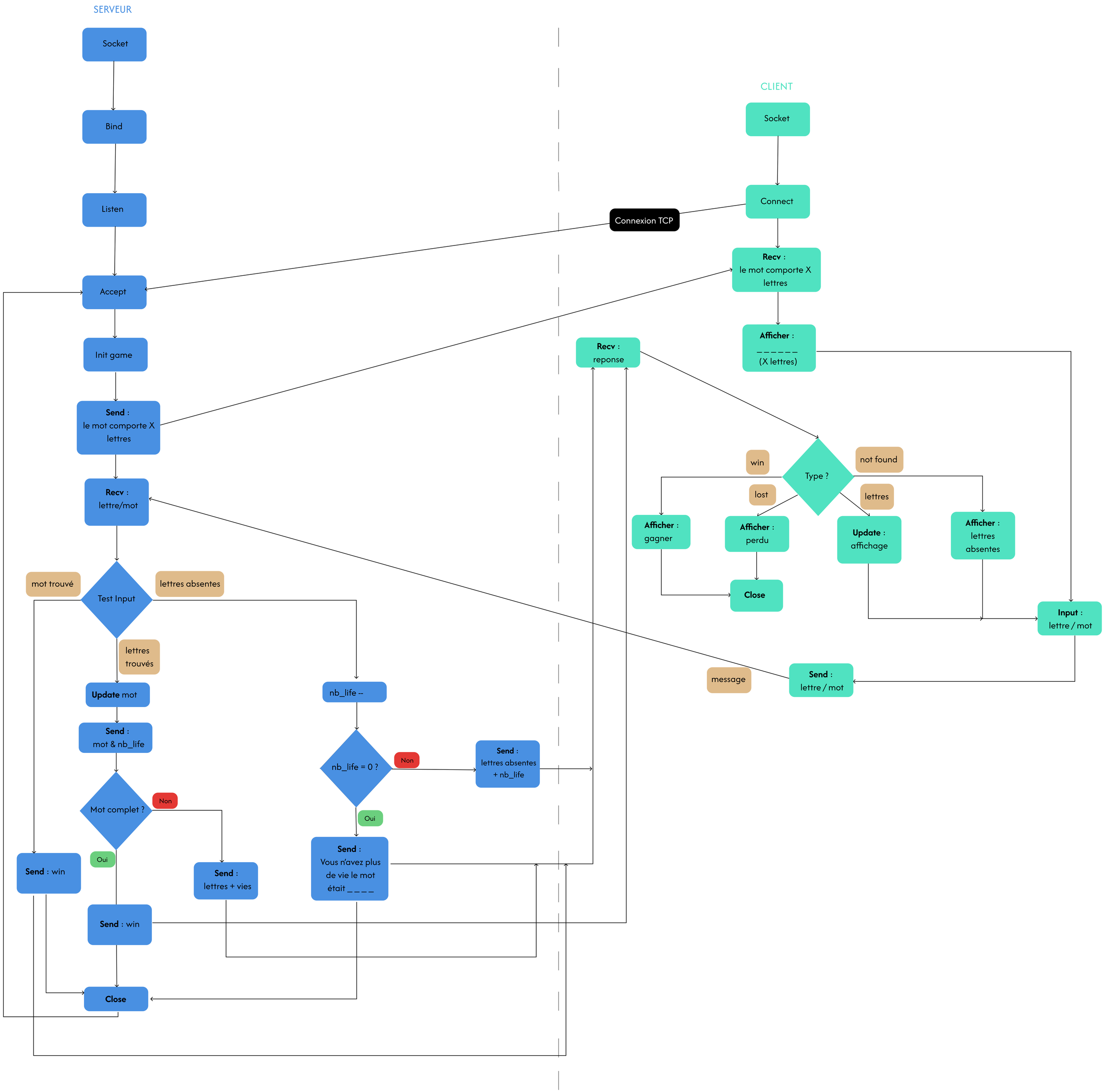
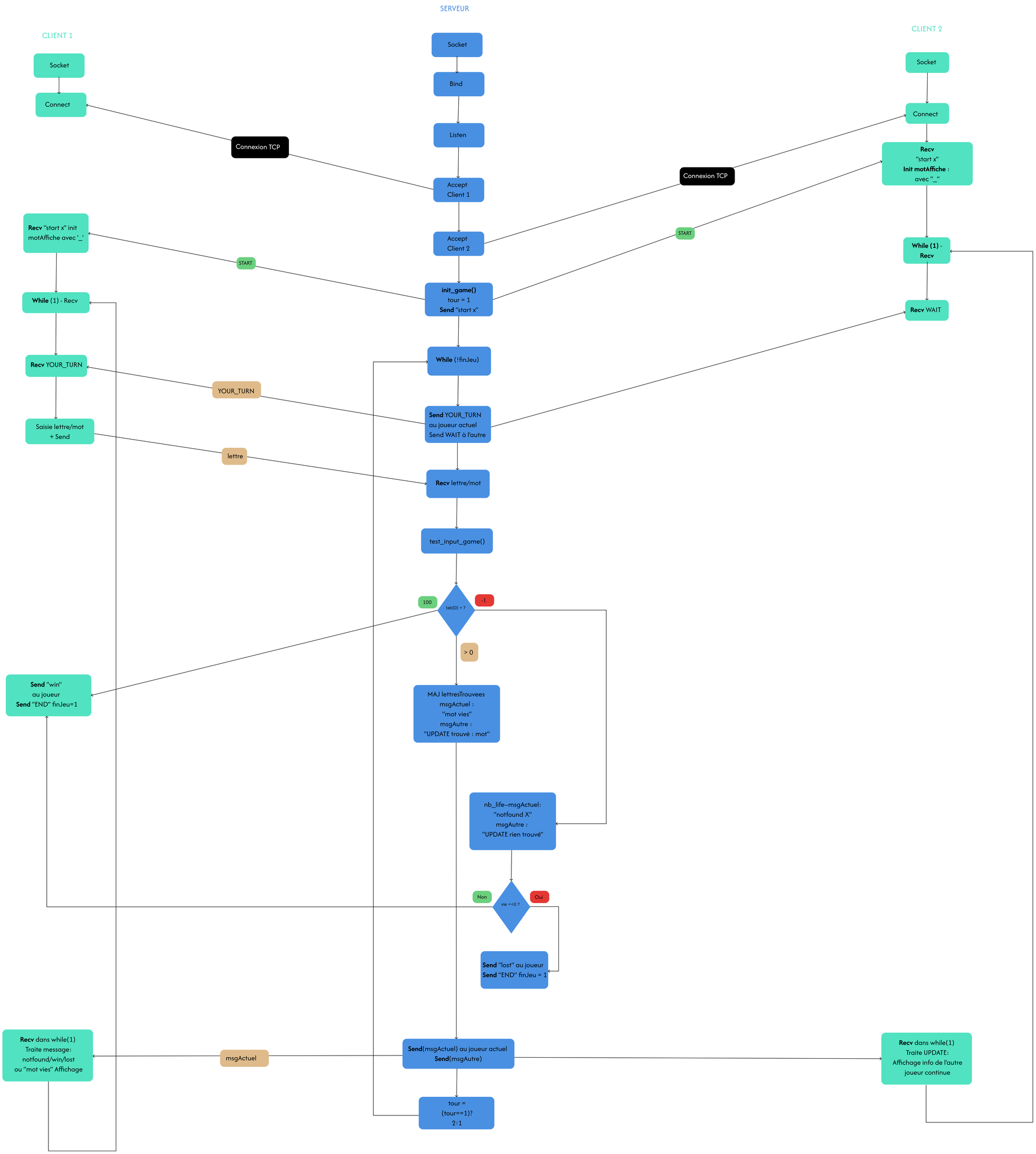
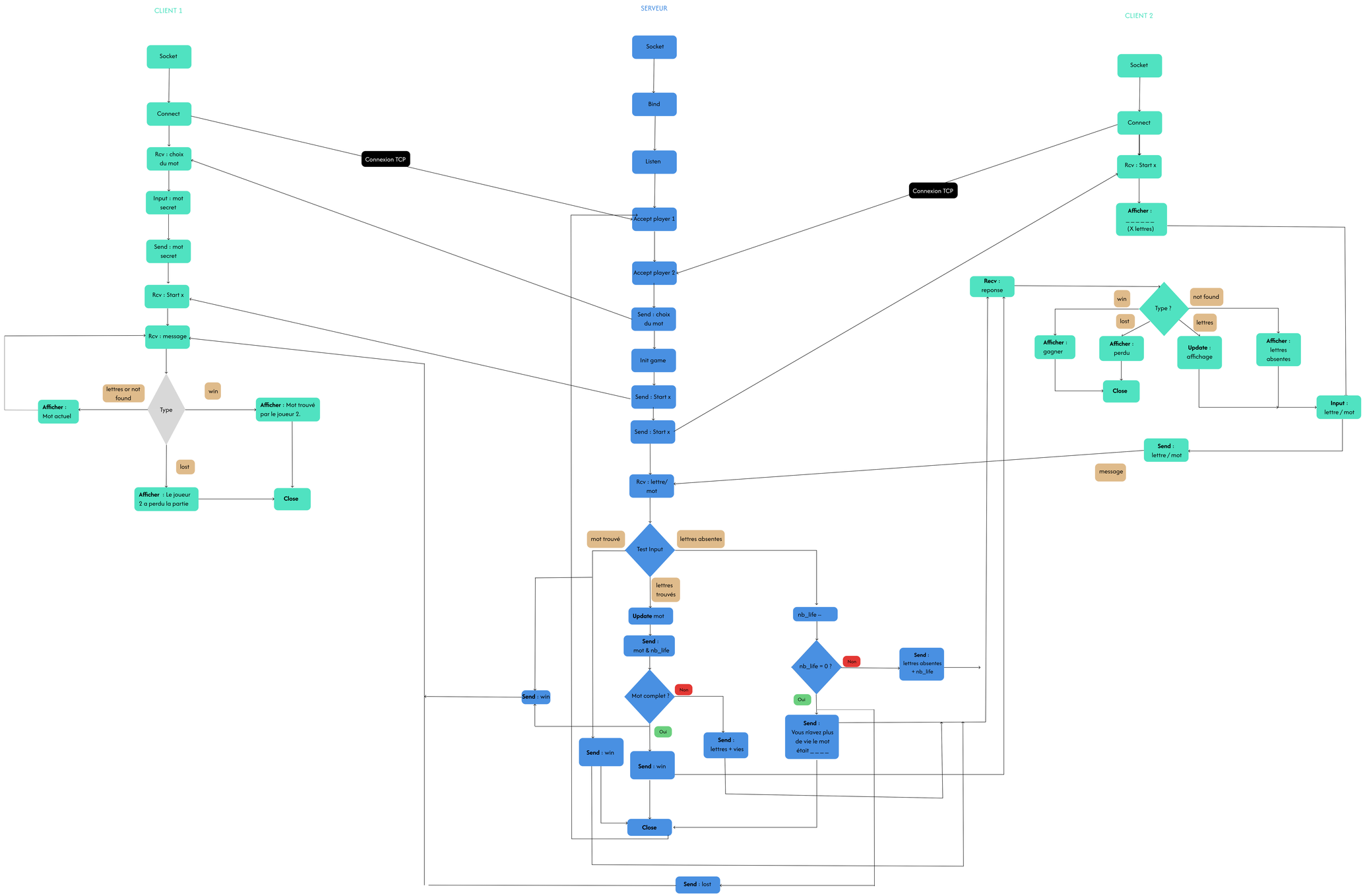
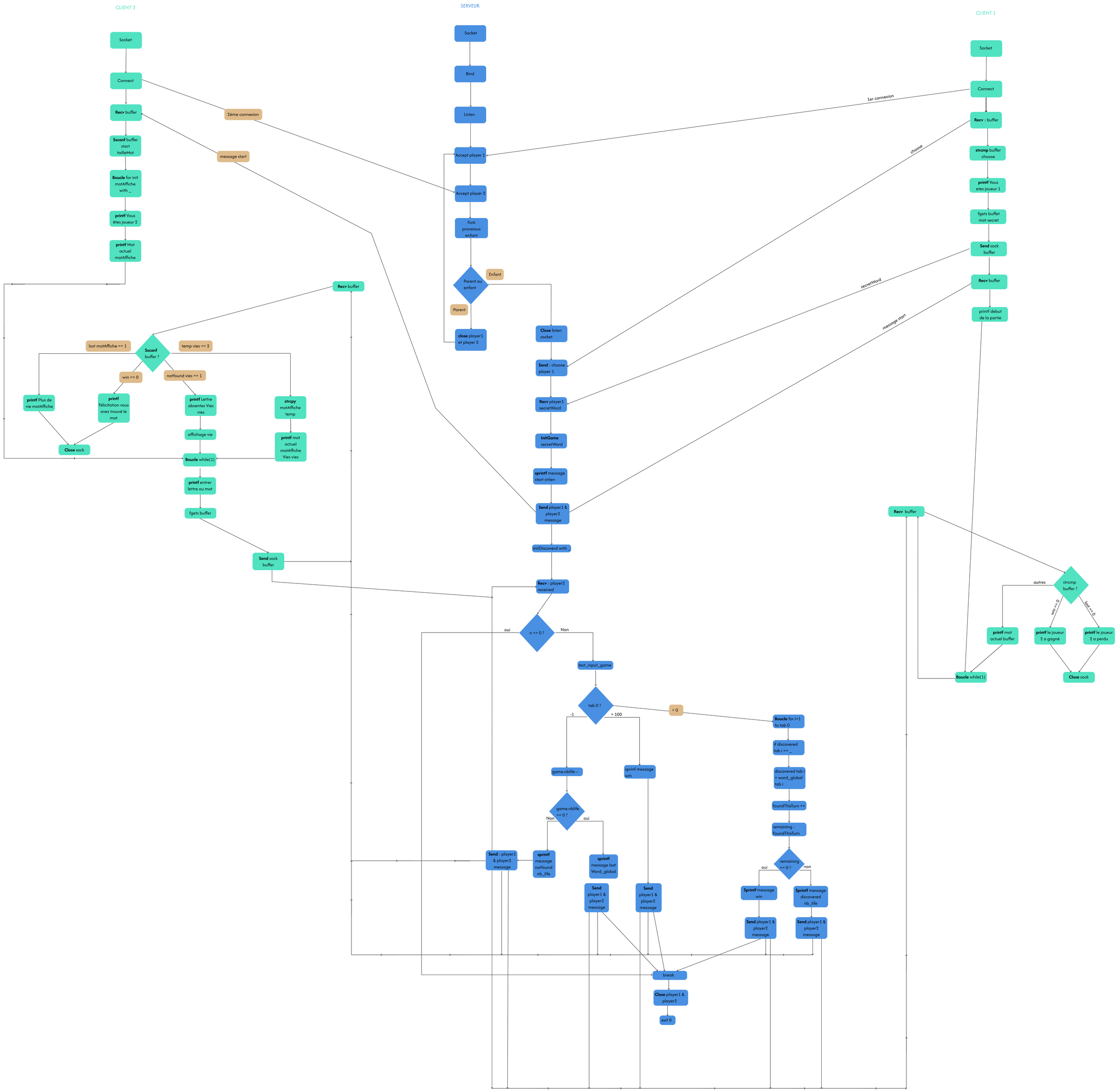
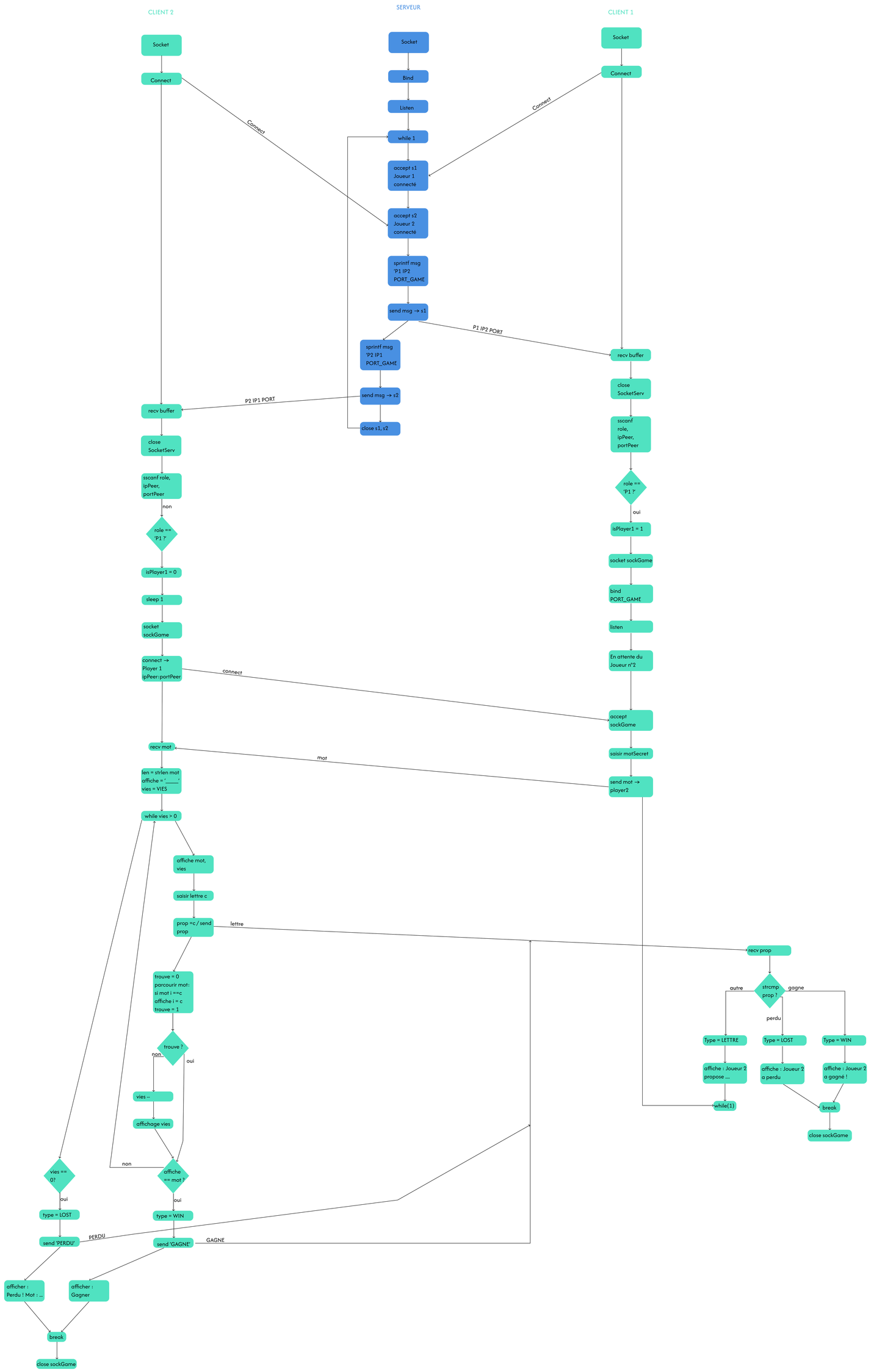

SERVEUR
Socket
Bind
Listen
Accept
Init game
Send : le mot comporte X lettres
Recv : lettre/mot
Test Input
mot trouvé
lettres absentes
lettres trouvés
Update mot
Send : mot & nb_life
Mot complet ?
Non
Oui
Send : win
Send : lettres + vies
Send : win
Close
nb_life --
nb_life = 0 ?
Non
Oui
Send : lettres absentes + nb_life
Send : Vous n'avez plus de vie le mot était _ _ _ _
Connexion TCP
CLIENT
Socket
Connect
Recv : le mot comporte X lettres
Afficher : _ _ _ _ _ _ (X lettres)
Recv : reponse
Type ?
win
lost
not found
lettres
Afficher : gagner
Afficher : perdu
Update : affichage
Afficher : lettres absentes
Close
Input : lettre / mot
message
Send : lettre / mot


SERVEUR
CLIENT 1
CLIENT 2
Socket
Bind
Listen
Accept Client 1
Accept Client 2
init_game() tour = 1 Send "start x"
While (!finJeu)
Send YOUR_TURN au joueur actuel Send WAIT à l'autre
Recv lettre/mot
test_input_game()
tab[0] = ?
100
-1
> 0
MAJ lettresTrouvees msgActuel : "mot vies" msgAutre : "UPDATE trouvé : mot"
nb_life--msgActuel: "notfound X" msgAutre : "UPDATE rien trouvé"
vie =<0 ?
Non
Oui
Send "lost" au joueur Send "END" finJeu = 1
Send(msgActuel) au joueur actuel Send(msgAutre)
tour = (tour==1)? 2:1
msgActuel
Recv dans while(1) Traite message: notfound/win/lost ou "mot vies" Affichage
Recv dans while(1) Traite UPDATE: Affichage info de l'autre joueur continue
Socket
Connect
Connexion TCP
Recv "start x" init motAffiche avec '_'
START
While (1) - Recv
Recv YOUR_TURN
YOUR_TURN
Saisie lettre/mot + Send
lettre
Send "win" au joueur Send "END" finJeu=1
Socket
Connect
Connexion TCP
Recv "start x" Init motAffiche : avec "_"
START
While (1) - Recv
Recv WAIT


CLIENT 1
SERVEUR
CLIENT 2
Socket
Connect
Rcv : choix du mot
Input : mot secret
Send : mot secret
Rcv : Start x
Rcv : message
Type
lettres or not found
win
lost
Afficher : Mot actuel
Afficher : Mot trouvé par le joueur 2.
Afficher : Le joueur 2 a perdu la partie
Close
Connexion TCP
Socket
Bind
Listen
Accept player 1
Accept player 2
Send : choix du mot
Init game
Send : Start x
Send : Start x
Rcv : lettre/ mot
Test Input
mot trouvé
lettres absentes
lettres trouvés
Update mot
Send : mot & nb_life
Mot complet ?
Non
Oui
Send : win
Send : lettres + vies
Send : win
Send : win
Close
nb_life --
nb_life = 0 ?
Non
Oui
Send : lettres absentes + nb_life
Send : Vous n'avez plus de vie le mot était _ _ _ _
Send : lost
Connexion TCP
Socket
Connect
Rcv : Start x
Afficher : _ _ _ _ _ _ (X lettres)
Recv : reponse
Type ?
win
lost
not found
lettres
Afficher : gagner
Afficher : perdu
Update : affichage
Afficher : lettres absentes
Close
Input : lettre / mot
Send : lettre / mot
message


CLIENT 2
SERVEUR
CLIENT 1
Socket
Connect
Recv buffer
Sscanf buffer start tailleMot
Boucle for init motAffiche with _
printf Vous êtes joueur 2
printf Mot actuel motAffiche
2ème connexion
message start
Socket
Bind
Listen
Accept player 1
Accept player 2
Fork processus enfant
Parent ou enfant
Enfant
Parent
close player1 et player 2
Socket
Connect
Recv : buffer
strcmp buffer choose
printf Vous etes joueur 1
fgets buffet mot secret
Send sock buffer
Recv buffer
printf debut de la partie
1er connexion
choose
secretWord
message start
Recv buffer
Sscanf buffer ?
lost motAffiche == 1
temp vies == 2
win == 0
notfound vies == 1
printf Plus de vie motAffiche
printf Félicitation vous avez trouvé le mot
printf Lettre absentes Vies vies
strcpy motAffiche temp
affichage vie
printf mot actuel motAffiche Vies vies
Close sock
Boucle while(1)
printf entrer lettre ou mot
fgets buffer
Send sock buffer
Close listen socket
Send : choose player 1
Recv player1 secretWord
InitGame secretWord
sprintf message start strlen
Send player1 & player2 message
initDiscoverd with _
Recv : player2 received
n <= 0 ?
oui
Non
test_input_game
tab 0 ?
-1
= 100
> 0
game.nblife --
sprinf message win
game.nblife <= 0 ?
Non
oui
Send : player1 & player2 message
sprintf message notfound nb_life
sprintf message lost Word_global
Send player1 & player2 message
Send player1 & player2 message
Boucle for i=1 to tab 0
if discovered tab i == _
discovered tab i = word_global tab i
foundThisTurn ++
remaining - foundThisTurn
remaining == 0 ?
oui
non
Sprintf message win
Sprintf message discovered nb_life
Send player1 & player2 message
Send player1 & player2 message
break
Close player1 & player2
exit 0
Recv buffer
strcmp buffer ?
autres
win == 0
lost == 0
printf mot actuel buffer
printf le joueur 2 a gagné
printf le joueur 2 a perdu
Boucle while(1)
Close sock


CLIENT 2
SERVEUR
CLIENT 1
Socket
Connect
Socket
Bind
Listen
while 1
accept s1
Joueur 1
connecté
accept s2
Joueur 2
connecté
sprintf msg
'P1 IP2
PORT_GAME
send msg → s1
sprintf msg
'P2 IP1
PORT_GAME
send msg → s2
close s1, s2
Connect
Connect
Socket
Connect
P1 IP2 PORT
recv buffer
close
SocketServ
sscanf
role,
ipPeer,
portPeer
role ==
'P1 ?'
oui
isPlayer1 = 1
socket sockGame
bind
PORT_GAME
listen
En attente du
Joueur n°2
accept
sockGame
saisir motSecret
send mot →
player2
P2 IP1 PORT
recv buffer
close
SocketServ
sscanf role,
ipPeer,
portPeer
non
role ==
'P1 ?'
isPlayer1 = 0
sleep 1
socket
sockGame
connect →
Player 1
ipPeer:portPeer
connect
recv mot
mot
len = strlen mot
affiche = '_____'
vies = VIES
while vies > 0
affiche mot,
vies
saisir lettre c
prop =c / send
prop
lettre
recv prop
trouve = 0
parcourir mot :
si mot i ==c
affiche i = c
trouve = 1
trouve ?
non
oui
vies --
affichage vies
affiche
== mot ?
non
oui
vies ==
0?
oui
type = LOST
send 'PERDU'
PERDU
type = WIN
send 'GAGNE'
GAGNE
afficher :
Perdu ! Mot : ...
afficher :
Gagner
break
close sockGame
strcmp
prop ?
autre
gagne
perdu
Type = LETTRE
Type = LOST
Type = WIN
affiche : Joueur 2
propose ....
affiche : Joueur 2
a perdu
affiche : Joueur 2
a gagné !
while(1)
break
close sockGame